

PRODUCT REQUIREMENTS DOCUMENT

Project Management App

Capstone Project - MVP

Version	1.0
Status	Approved for Design Phase
Team Size	4 members
Prepared For	Capstone Project Team
Date	25 September 2026

Source basis: TS Academy Capstone Project instructions, the team MVP specification, and decisions agreed during product planning.

Document Control

Item	Details
Document purpose	Define what the MVP must do, who it is for, how users are allowed to interact with it, and how the team will know it is complete.
Product	Project Management App
MVP principle	Build the smallest complete product that lets a real user complete the core workflow from beginning to end.
Primary audience	Students, startups/small companies, freelancers, and small businesses working in small collaborative teams.
Next phase after approval	Database design, API design, UI/wireframes, GitHub setup, then task distribution.

PRD Contents

- 1. Product Overview
 - 1.1 Technology Stack
- 2. Problem Statement
- 3. Target Users
- 4. Product Goals
- 5. User Roles and Permissions
- 6. Core User Flows
- 7. MVP Scope and Features
- 8. Functional Requirements
- 9. Business Rules
- 10. Non-Functional / Capstone Requirements
- 11. Out of Scope
- 12. Success and Acceptance Criteria
- 13. Open Implementation Decisions
- 14. Appendix A. Team Lead Review Checklist

PRD rule: When a new feature idea appears, first ask whether it is required for the core user journey. If not, keep it outside the MVP unless the team formally updates this document.

1. Product Overview

The Project Management App is a collaborative full-stack web application that allows users to create projects, invite collaborators, create and assign tasks, set deadlines and statuses, and track project progress.

The MVP is intended to demonstrate a complete working product, not a commercial system with every possible project-management feature.

1.1 Technology Stack

The MVP will use the following project stack. It follows the capstone recommendation of React for the frontend, Node.js + Express.js for the backend, and MongoDB for a simple database setup, while adding the supporting tools needed for authentication, email, collaboration, documentation, and source control.

Layer / Area	Selected Technology	Purpose in the Project
Frontend	React.js	Build the user interface, dashboard, project screens, task screens, forms, and role-based views.
Client-side routing	React Router	Handle navigation between public pages, authentication pages, dashboard, projects, and task views.
API communication	Axios	Send requests from the React frontend to the Express REST API and handle responses/errors.
Frontend state	Context API	React's built-in way to share state/data across multiple components without passing props through every intermediate component. Use it for shared data such as the logged-in user, authentication state, and other app-wide state where appropriate.
Styling	CSS / Tailwind CSS	Create a usable and responsive interface. The team should settle on one styling approach before UI implementation.
Backend	Node.js + Express.js	Provide REST APIs, business logic, authorization, validation, error handling, and database operations.
Database	MongoDB + Mongoose	Store users, projects, project membership, tasks, assignments, deadlines, and related application data.
Authentication & security	JWT + bcrypt	Authenticate users, protect private routes, authorize project actions, and securely hash passwords.
Email	Nodemailer + SMTP/email provider	Send email verification, welcome, password-reset, and overdue-reminder messages. The final provider/transport remains a technical setup decision.

Layer / Area	Selected Technology	Purpose in the Project
Environment configuration	dotenv	Keep database URLs, JWT secrets, email credentials, and other sensitive configuration outside source code.
API documentation & manual testing	Swagger / OpenAPI + Postman	Use Swagger/OpenAPI for interactive API documentation and endpoint schemas, and Postman for manual API testing, saved requests, example responses, and team collections.
Version control & collaboration	Git + GitHub	Use feature branches, pull requests, reviews, and a shared repository for the four-person team.
Deployment	Platform to be selected during deployment phase	Deploy the frontend, backend, and database after integration and testing; the exact hosting services are not fixed by the PRD yet.

Stack note: React, Node.js + Express.js, and MongoDB come directly from the capstone guidance. Supporting choices such as Axios, Context API, Mongoose, JWT, bcrypt, Nodemailer, dotenv, Swagger/OpenAPI, Postman, Git, and GitHub define how this team intends to implement the MVP. Hosting and the exact email provider remain implementation decisions.

2. Problem Statement

Students, startups, freelancers, and small businesses often work on projects involving multiple people, tasks, responsibilities, and deadlines. Without a simple shared system, it can become difficult to know who is responsible for each task, what work is completed, what remains pending, and whether deadlines are being met.

The product will provide one shared place where small teams can organize projects, assign responsibilities, manage deadlines, update task status, and see progress.

3. Target Users

Primary target: small collaborative teams that need a simple way to organize work and track responsibility.

- Students working on group assignments, capstone projects, clubs, or school activities.
- Startups and small companies coordinating work across small teams.
- Freelancers working with partners, clients, or other freelancers on shared work.
- Small businesses managing internal projects, campaigns, events, or operational tasks.

4. Product Goals

- Allow a user to create and organize a project and the people working on it.
- Make responsibilities clear by assigning tasks to one or more project members.
- Help team members understand the current state of work through task status and project progress.
- Help teams manage time through project and task deadlines.
- Keep the experience simple enough for small teams without enterprise-level complexity.
- Provide a complete end-to-end workflow that connects frontend, backend, database, authentication, authorization, and email services.

5. User Roles and Permissions

Roles are project-specific. A user may be the Admin/Owner of one project and a Collaborator in another. Creating an account does not permanently make a user an Admin or Collaborator.

Role	How role is obtained	Core responsibility
Admin / Project Owner	Creates the project.	Controls shared project management: project settings, collaborators, tasks, assignments, deadlines, and project-wide progress.
Collaborator / Invitee	Is added/invited to a project by the Admin.	Participates in the project, sees project information, works on assigned tasks, updates permitted task status, and can leave the project.

Action	Admin	Collaborator
Create a project	Yes	Yes
View a project they belong to	Yes	Yes
Update shared project details	Yes	No
Delete shared project	Yes	No
Add/remove collaborators	Yes	No
Create/update/delete shared tasks	Yes	No (except permitted status update)
Assign/reassign task members	Yes	No
Set/change deadlines	Yes	No
View all project tasks	Yes	Yes
Update status of assigned task	Yes	Yes
Track project progress	Yes	Yes
Leave project	Not the normal owner action	Yes

Shared-interface principle: Admins and Collaborators may view the same core project information and screens, but the actions shown and allowed depend on the user's role and task assignment. Backend authorization must enforce the rule even if a frontend button is hidden.

6. Core User Flows

6.1 Main Admin / Owner Flow

1. Register an account.
2. Verify the registered email address.
3. Receive a welcome/confirmation email after successful verification.
4. Log in and reach the dashboard.
5. Create a project and automatically become that project's Admin/Owner.
6. Add collaborators by username.
7. Create tasks inside the project.
8. Assign each task to one or more project members; the Admin may also assign themselves.
9. Set project and task deadlines.
10. Monitor task statuses and overall project progress.
11. Update project/task information when permitted and remove collaborators when necessary.
12. Complete or delete the project when appropriate.

6.2 Collaborator Flow

1. Log in to an existing verified account.
2. See projects in which the user is a collaborator.
3. Open a project and view the project information and all project tasks.
4. Identify tasks assigned to them.
5. Update the status of an assigned task from To Do to In Progress to Completed.
6. View deadlines and overall progress.
7. Leave the project when they no longer need to participate; leaving must not delete the shared project.

6.3 Password Recovery Flow

1. User selects Forgot Password.
2. User submits their registered email address.
3. System sends a secure password-reset email/link or token.
4. User sets a new password using the valid reset flow.
5. User can log in with the new password.

6.4 Core Workflow Summary

End-to-end workflow: Register -> Verify Email -> Login -> Create Project -> Add Collaborators -> Create Tasks -> Assign Tasks -> Set Deadlines -> Update Status -> Track Progress -> Complete Project

7. MVP Scope and Features

Feature area	MVP capability
Authentication	Registration, email verification, welcome email, login, forgot password, password reset, protected routes.

Feature area	MVP capability
User identity	Full name or company name, unique username, email, password.
Projects	Create, view, update, and delete with role-based authorization.
Collaboration	Admin adds collaborators by username; Admin can remove them; Collaborator can leave a project.
Tasks	Admin creates, views, updates, and deletes tasks; Collaborators can view all project tasks.
Assignments	Admin assigns one or multiple project members to a task; Admin may assign themselves.
Deadlines	Admin sets project/task deadlines; overdue items are visibly marked.
Task status	To Do, In Progress, Completed.
Progress	Automatically calculated from completed tasks relative to total tasks.
Project status	Derived from progress: Active, Almost Done, Completed.
Email	Verification, welcome confirmation, password reset, overdue reminder emails.
Dashboard	Shows projects the user owns or collaborates on, with role-appropriate actions.

8. Functional Requirements

The following requirements use unique IDs so the team can reference them during design, development, pull-request review, and testing.

Authentication and account

ID	Requirement
FR-01	A new user shall be able to register using full name/company name, username, email, and password.
FR-02	The username shall uniquely identify one account because collaborators are added by username.
FR-03	After registration, the system shall send an email-verification message.
FR-04	The user shall verify the registered email address before the account is treated as fully active.
FR-05	After successful verification, the system shall send a welcome/confirmation email.
FR-06	A verified user shall be able to log in with valid credentials.
FR-07	Only authenticated users shall access private project-management functionality.
FR-08	A user shall be able to request a password-reset email and set a new password through a secure reset process.

Project management

ID	Requirement
FR-09	Any authenticated user shall be able to create a project.
FR-10	The creator of a project shall automatically become that project's Admin/Owner.
FR-11	Users shall be able to view projects they own or projects in which they are collaborators.
FR-12	Only the Project Admin shall update shared project information.
FR-13	Only the Project Admin shall permanently delete the shared project.

Collaboration

ID	Requirement
FR-14	The Project Admin shall be able to add an already registered user to the project using that user's username.

ID	Requirement
FR-15	The Project Admin shall be able to remove a collaborator from the project.
FR-16	Removing a collaborator from the project shall also remove that user from task assignments within the same project.
FR-17	Removing a user from one task shall not remove the user from the project.
FR-18	A Collaborator shall be able to leave a project without deleting the shared project for other members.

Task management and assignment

ID	Requirement
FR-19	The Project Admin shall be able to create tasks inside the project.
FR-20	Project members shall be able to view all tasks in projects they belong to.
FR-21	The Project Admin shall be able to update task details and delete shared tasks.
FR-22	The Project Admin shall be able to assign one task to one or more project members.
FR-23	The Project Admin shall be able to assign themselves to a task.
FR-24	Only users who belong to the project may be assigned to the project's tasks.
FR-25	Collaborators shall not assign tasks to other users or change task assignees.

Status, deadlines, and progress

ID	Requirement
FR-26	Every task shall use one of the MVP statuses: To Do, In Progress, or Completed.
FR-27	A Collaborator shall be able to update the status of a task only when they are one of its assignees.
FR-28	The Project Admin shall be able to set and change project and task deadlines.
FR-29	Collaborators shall be able to view deadlines but shall not modify them.
FR-30	An incomplete task or project whose deadline has passed shall be displayed as Overdue.

ID	Requirement
FR-31	The system shall send an email reminder for an overdue task/project according to the agreed reminder schedule.
FR-32	Project progress shall be calculated automatically as completed tasks divided by total tasks, multiplied by 100.
FR-33	Project status shall be derived from progress: Active from 0-74%, Almost Done from 75-99%, Completed at 100%.

Dashboard and shared interface

ID	Requirement
FR-34	After login, a user shall see projects they own and projects in which they are a collaborator.
FR-35	Admins and Collaborators may access the same core project views, but available actions shall depend on project role and assignment.
FR-36	The frontend shall not rely on hidden buttons as the only security measure; unauthorized backend requests must also be rejected.

Validation and user feedback

ID	Requirement
FR-37	The backend shall validate required fields, email values, IDs, dates, and other incoming request data.
FR-38	The application shall provide understandable error messages instead of exposing raw internal errors.
FR-39	The frontend shall provide a loading state while relevant API requests are being processed.

9. Business Rules

ID	Rule
BR-01	Project roles are project-specific; a user can be an Admin in one project and a Collaborator in another.
BR-02	Project ownership is created automatically when a user creates a project.
BR-03	Deleting a project is a shared destructive action and is Admin-only.
BR-04	Leaving a project is a collaborator self-action and must not delete the shared project.
BR-05	A task may have one or multiple assignees.
BR-06	Only project members may be assigned to that project's tasks.
BR-07	Removing a collaborator from a project removes them from all task assignments in that project.
BR-08	Removing someone from one task does not remove them from the project.
BR-09	Collaborators can see all project tasks but may only perform collaborator-permitted actions.
BR-10	A collaborator may update task status only when assigned to that task.
BR-11	Task statuses are limited to To Do, In Progress, and Completed for the MVP.
BR-12	Project progress is system-calculated, not manually typed by users.
BR-13	Project status is automatically derived from project progress: Active 0-74%, Almost Done 75-99%, Completed 100%.
BR-14	Overdue is a deadline condition, not a replacement for the task's workflow status.
BR-15	Admins and Collaborators may share the same project UI, but actions are role-based and backend-enforced.
BR-16	Username must be unique because collaborator invitations use usernames.

9.1 Project Progress Calculation

Formula: Project Progress (%) = (Completed Tasks / Total Tasks) x 100. If a project has no tasks yet, the UI should display 0% rather than attempt division by zero.

Example	Result
10 total tasks, 5 completed	50% - Active
20 total tasks, 15 completed	75% - Almost Done
8 total tasks, 8 completed	100% - Completed

10. Non-Functional / Capstone Requirements

Area	Requirement
Frontend	React-based usable and responsive interface. Pages must form one connected product and display real backend data.
Backend	Node.js + Express.js REST API with business logic, database operations, validation, authorization, and error handling.
Database	Use properly separated major entities rather than one giant collection/table. MongoDB is suitable for the MVP.
Security	Hash passwords; use JWT or another suitable authentication mechanism; protect private routes; validate input; do not expose passwords or secrets.
Environment configuration	Use environment variables for secrets/configuration and do not commit .env.
API consistency	Use a consistent API response structure across endpoints.
API documentation	Use both Swagger/OpenAPI and Postman. Document endpoint, HTTP method, purpose, authentication, request body/parameters, successful response, and error response.
Testing	Perform basic testing of the complete user journey and permission boundaries.
Deployment	Deploy a working frontend/backend/database configuration that can be demonstrated.
Git workflow	Use feature branches and pull requests. Team members should not develop directly on main.

11. Out of Scope for MVP

The following features are intentionally excluded from the current MVP unless the team formally changes the PRD:

- Real-time team chat.
- Voice or video meetings.
- General-purpose file storage or cloud drive features.
- AI task suggestions or AI project planning.
- Advanced analytics and reporting dashboards.
- Time tracking and timesheets.
- Billing, subscriptions, or payment processing.
- Calendar integrations with third-party services.
- Push notifications, SMS, or a full notification-preference center.
- Complex organization/company hierarchy and enterprise administration.
- Native mobile applications.
- Real-time collaborative document editing.
- Advanced automation rules and external integrations.
- Advanced audit-log infrastructure.

Important: Email is NOT out of scope. The MVP includes email verification, welcome confirmation, forgot-password/reset, and overdue reminder emails.

12. Success and Acceptance Criteria

ID	Acceptance criterion
AC-01	A new user can register, receive verification email, verify the account, receive a welcome email, and log in.
AC-02	A user can complete the forgot-password and password-reset flow by email.
AC-03	A logged-in user can create a project and automatically become its Admin.
AC-04	The Admin can add a registered collaborator by username.
AC-05	The Admin can create a task and assign one or multiple project members, including themselves.
AC-06	A collaborator can see all project tasks but cannot perform Admin-only task or project actions.
AC-07	An assigned collaborator can update their task status and the change is visible to the team.
AC-08	Project progress changes automatically when tasks are completed and project status follows the agreed thresholds.
AC-09	Past-deadline incomplete work is visibly marked Overdue and the required reminder email is sent according to the configured schedule.

ID	Acceptance criterion
AC-10	Removing a collaborator from a project removes them from that project's task assignments but does not delete other users or tasks.
AC-11	Removing a user from a single task leaves the user as a project member.
AC-12	A collaborator can leave a project without deleting it for remaining members.
AC-13	Unauthorized backend actions are rejected even if someone manually calls the API.
AC-14	The frontend displays real API/database data and provides loading/error feedback.
AC-15	The API is documented, core flows are tested, and the product is deployed for demonstration.

12.1 Final End-to-End Test

1. Create User A and User B accounts and verify both emails.
2. Log in as User A and create a project.
3. Add User B as a collaborator by username.
4. Create several tasks and assign User B to at least one task; assign multiple users to another task.
5. Set project/task deadlines.
6. Log in as User B, open the project, view all tasks, and update the status of an assigned task.
7. Confirm User B cannot delete/update the shared project, assign users, or change deadlines.
8. Log in as User A and confirm User B's status update and the recalculated project progress.
9. Remove User B from one task and confirm User B remains a project member.
10. Remove User B from the project and confirm User B is also removed from that project's task assignments.
11. Test forgot-password flow, validation errors, loading states, and at least one overdue reminder scenario.

13. Open Implementation Decisions

The product rules above are approved for PRD v1.0. The following details are still implementation-level decisions and should be resolved during database/API/UI design rather than guessed by individual developers:

Decision	Why it still matters
Invitation acceptance	Decide whether adding by username immediately adds the collaborator or whether the collaborator must accept an invitation.
Overdue email timing	Decide exactly when reminders are sent (for example, once after deadline, daily until completion, etc.). Keep the MVP simple.

Decision	Why it still matters
Password policy	Define minimum length and other validation rules before backend validation is implemented.
Username format	Define allowed characters, case handling, minimum/maximum length, and whether username changes are allowed.
Project/task fields	Finalize exact fields such as title, description, dates, optional notes, and whether project start date is required.
Deletion strategy	Decide hard delete vs soft delete for projects/tasks during technical design.
Email provider	Choose the email delivery service/library and environment-variable configuration.
UI design system	Choose the layout approach and CSS/UI library before frontend implementation begins.

Phase gate: Do not distribute major implementation work until the database design, API contract, and core UI/wireframes are agreed. These three designs should all reflect this PRD.

Appendix A - Team Lead Review Checklist

Use this checklist as the project moves from PRD into implementation. The lead does not need to know every answer personally; the lead must make sure the team reaches one shared answer and records it.

Checkpoint	Question for the team lead
Before database design	Can every team member explain the problem, target users, core workflow, roles, MVP scope, and business rules?
Database review	Does the data model support project-specific ownership, collaborators, multiple task assignees, deadlines, status, and leave/remove behaviour?
API review	Does every frontend action have a clear endpoint/response, and are Admin/Collaborator permissions explicit?
UI review	Can both roles understand the same project information while seeing only actions they are allowed to perform?
Task distribution	Does every team member have a clear deliverable, dependencies, branch, and definition of done?
Pull request review	Was the feature tested locally? Does it match the PRD/API contract? Are permission checks enforced in the backend?
Integration review	Do frontend, backend, database, and email flows work together rather than as isolated pieces?
MVP freeze	When someone proposes a new feature, does it support the core journey strongly enough to justify changing the PRD?
Final test	Can a brand-new user complete the end-to-end acceptance flow without manual database edits or developer intervention?
Technology stack review	Has the team agreed on the frontend, backend, database, authentication, email, API tooling, and version-control choices, and are any unresolved tools clearly marked as pending?

Source Documents Used

- TS Academy Capstone Project - Full-Stack MVP Requirements and project topic list.
- Capstone Project - Project Management App Core Workflow & MVP Role Permissions.
- Team planning decisions recorded during PRD discussions, including handwritten answers to open questions and the shared-interface rule.

PRD versioning: If the team changes a product rule after development begins, update the PRD version and record the decision before developers implement different interpretations.